Lab 01 — Praktijklab: isolatie met eigen ogen zien

Doel: elke bewering uit de theorie zelf experimenteel controleren. Aan het einde heb je gezien dat een container een proces van je WSL-machine is, en dat de `root` van een rootless container... jij bent.

Vereisten — Windows 10/11 met WSL 2 en een Ubuntu-distributie (22.04 of recenter). Voor dit lab zijn geen bestanden nodig. De getoonde uitvoer komt van Podman 5.8; vanaf Podman 4.9 zijn de commando's identiek, alleen kleine weergavedetails verschillen.

Stap 0 — WSL voorbereiden en Podman installeren

In een **PowerShell**-terminal (Windows):

```
wsl --version          # WSL version 2.x verwacht
wsl --list --verbose   # je Ubuntu moet VERSION 2 zijn
```

Daarna in de **Ubuntu**-terminal:

```
cat /etc/wsl.conf
```

Kijk na of het bestand `[boot]` bevat, gevolgd door `systemd=true`. Zo niet, voeg het toe:

```
printf '[boot]\nsystemd=true\n' | sudo tee /etc/wsl.conf
```

Doe vervolgens vanuit PowerShell `wsl --shutdown` en open Ubuntu opnieuw. Installeer daarna Podman:

```
sudo apt update && sudo apt install -y podman
podman --version
```

> **→ WINDOWS / WSL**
> WSL 2 is een piepkleine Hyper-V-VM die in één seconde opstart en RAM deelt met Windows. Standaard draait er **geen** `systemd` in — een historische keuze van Microsoft. En laat het nu net `systemd` zijn dat jouw gebruiker het recht geeft om *cgroups* aan te maken: zonder werken `podman run --memory` en `podman stats` niet in rootless-modus. Vandaar de stap hierboven. Docker Desktop of Podman Desktop heb je niet nodig: Podman is hier een gewoon Ubuntu-pakket. (Gebruik je toch Podman Desktop, dan maakt `podman machine` zijn eigen WSL-distributie aan; de commando's van dit lab blijven identiek.)

Stap 1 — De engine identificeren

```
podman version
```

Observeer één enkel blok, `Client: Podman Engine`, met `Version: 5.x.x` en `OS/Arch: linux/amd64`. Geen `Server`-blok.

```
podman info | head -n 40
podman info --format 'rootless={{.Host.Security.Rootless}} cgroups={{.Host.CgroupManager}}
netwerk={{.Host.NetworkBackend}} runtime={{.Host.OCIRuntime.Name}}'
```

Observeer `rootless=true cgroups=systemd netwerk=netavark runtime=crun`, en in de lange uitvoer de regels `kernel: 6.6.87.2-microsoft-standard-WSL2`, `idMappings:` (daarover meer in stap 5) en `graphRoot: /home/<jij>/.local/share/containers/storage`.

Uitleg. Bij Docker ondervraagt `version` twee helften — client en daemon — en beschrijft `info` de daemon. Bij Podman is er maar één programma: `podman info` beschrijft wat **jouw gebruiker** kan doen. De `graphRoot` in je eigen `home` bevestigt het: de images staan niet in `/var/lib`, ze zijn van jou.

Stap 2 — De eerste container, en waar hij naartoe is

```
podman run alpine echo "hallo vanuit de container"
```

Observeer eerst `Resolved "alpine" as an alias (/etc/containers/registries.conf.d/000-shortnames.conf)`, daarna `Trying to pull docker.io/library/alpine:latest...`, enkele regels `Copying blob`, `Writing manifest`, de afgedrukte boodschap… en meteen daarna sta je weer aan de prompt.

> **PODMAN**
> Docker vult `alpine` stilzwijgend aan tot `docker.io/library/alpine`. Podman weigert te gokken: het kijkt in een lijst met gekende aliassen (`alpine`, `nginx`, `debian`, `node`, `postgres`…) en bij een onbekende naam **vraagt** het je op welke registry het moet zoeken — of het faalt als er geen terminal beschikbaar is. Daarom staat in Dockerfiles en scripts van bedrijven altijd de volledige naam: `docker.io/library/eclipse-temurin:21-jre`. Maak er nu al een gewoonte van.

```
podman ps
podman ps -a
```

Observeer dat `podman ps` **niets** toont, maar dat `podman ps -a` de container wél toont: met een willekeurige naam (`trusting_sanderson`…), de image onder haar volledige naam `docker.io/library/alpine:latest`, en de status `Exited (0)`.

```
podman run --rm alpine echo "deze laat geen spoor na"
podman ps -a
```

Observeer dat er geen nieuwe container bijkomt: `--rm` ruimt de container op zodra hij stopt.

Uitleg. Een container leeft precies zo lang als zijn hoofdproces. `echo` schreef één regel en stopte; de container stierf mee, maar werd niet verwijderd — hij blijft achter als een lijk dat je nog kunt inspecteren. `podman ps` toont alleen draaiende containers.

Stap 3 — De kernel is die van de host (en de host is WSL)

```
uname -r
podman run --rm alpine uname -r
podman run --rm debian uname -r
```

Observeer dat alle **drie** de commando's dezelfde waarde tonen, bijvoorbeeld `6.6.87.2-microsoft-standard-WSL2` — en dat terwijl Ubuntu, Alpine en Debian drie verschillende systemen zijn.

```
podman run --rm alpine cat /etc/os-release | head -n 2
podman run --rm debian cat /etc/os-release | head -n 2
```

Observeer dit keer twee verschillende resultaten: `Alpine Linux` en `Debian GNU/Linux`.

Uitleg. Daarmee is het bewezen: de image levert de *userland* (bestanden, binaries, bibliotheken), de kernel komt van de host en wordt nooit gedupliceerd. En die host is niet Windows: het achtervoegsel `microsoft-standard-WSL2` is de handtekening van de Linux-kernel die Microsoft voor WSL compileert. Je containers draaien in die VM.

> **→ LINUX**
> `/etc/os-release` is een gewoon tekstbestand dat elke distributie meelevert om zich voor te stellen. `uname -r` is daarentegen een **systeemaanroep**: het antwoord komt van de kernel. Daarom verschilt het eerste van container tot container, en het tweede niet.

Stap 4 — Het proces van beide kanten bekijken

Start een container die een tijdje blijft draaien:

```
podman run -d --name waker alpine sleep 600
podman ps
```

Observeer de status `Up`, de naam `waker` en het commando `sleep 600`.

Het zicht **van binnenuit**:

```
podman exec waker ps -o pid,ppid,comm
```

Observeer een piepkleine lijst: `sleep` heeft **PID 1**, en je `ps` PID 2.

Het zicht **vanaf de host**:

```
ps -ef | grep "[s]leep 600"
podman inspect --format '{{.State.Pid}}' waker
```

Observeer dat hetzelfde proces op de host bestaat, eigendom van **jouw gebruiker**, met een gewone PID (bijvoorbeeld `1854`) — en dat `podman inspect` je precies die PID geeft.

```
podman top waker
```

Observeer `USER root`, `PID 1`, `COMMAND sleep 600`: het "containerzicht" op hetzelfde proces, door Podman gereconstrueerd.

Uitleg. Eén en hetzelfde proces, twee nummeringen. Binnenin laat de `pid`-namespace het geloven dat het het eerste proces van het systeem is; buiten is het één proces tussen honderden andere — en het is van jou. Dat is het hele idee achter een container.

Ga na dat je die isolatie ook kunt weglaten:

```
podman run --rm --pid=host alpine ps -o pid,comm | head -n 8
```

Observeer de processen van **je WSL** (`init`, `systemd`, `conmon`…), opgelijst vanuit een container.

Uitleg. Isolatie is een optie, geen ingebakken eigenschap. Daarom zijn `--pid=host` en `--privileged` in productie standaard verboden. Merk trouwens `conmon` op: dat is het kleine toezichtsproces dat Podman bij elke container achterlaat, want een daemon om dat te doen is er niet.

Stap 5 — De root die er geen is (rootless)

```
podman exec waker id
```

Observeer `uid=0(root) gid=0(root)`: binnen de container draait `sleep` als root.

```
podman top waker user,huser,pid,hpid,comm
```

Observeer:

```
USER        HUSER       PID         HPID        COMMAND
root        1000        1           1854        sleep 600
```

`USER` is de identiteit zoals de container ze ziet, `HUSER` de echte identiteit op de host — en `1000`, dat ben jij (controleer met `id -u`).

```
podman unshare cat /proc/self/uid_map
```

Observeer een vertaaltabel in deze vorm:

```
         0       1000          1
         1     100000      65536
```

Uitleg. Dit is de `user`-namespace in actie. Regel 1: UID `0` van de container **is** jouw UID `1000`. Regel 2: de UID's `1` tot `65536` van de container worden afgebeeld op een "reservebereik" (`100000`+, vastgelegd in `/etc/subuid`) dat niemand anders gebruikt. Op de host heeft de "root" van de container dus alleen jouw rechten. Een gecompromitteerde container kan op je WSL geen root worden: er valt gewoon niets te escaleren.

→ BEVEILIGING
Bij Docker draait de daemon als root, en een container-`root` is — tenzij speciaal geconfigureerd — de echte root van de host. De isolatie steunt dan volledig op de `pid` / `mnt` / `net`-namespaces en op de weggenomen *capabilities*. Rootless Podman voegt een laag toe die Docker standaard niet heeft: zelfs als al de rest het begeeft, blijft de aanvaller een gewone gebruiker.

Stap 6 — Onveranderlijke image, wegwerpbare container

```
podman run -d --name c1 alpine sleep 600
podman run -d --name c2 alpine sleep 600
podman exec c1 sh -c 'echo "gegevens van c1" > /merk.txt'
```

Ga na dat schrijfacties geïsoleerd blijven:

```
podman exec c1 cat /merk.txt      # toont: gegevens van c1
podman exec c2 cat /merk.txt      # cat: can't open '/merk.txt': No such file or directory
```

Ga na dat de image zelf onaangeroerd bleef:

```
podman run --rm alpine ls /merk.txt    # No such file or directory
```

Meet die schrijflaag:

```
podman ps -s --format 'table {{.Names}}\t{{.Size}}'
```

Observeer een grootte als `11.4kB (virtual 8.72MB)`: `virtual` is image plus laag, en de eerste waarde is wat de container **zelf** verbruikt — een paar kilobytes metadata, plus jouw bestand.

Vernietig ten slotte en begin opnieuw:

```
podman rm -f -t 0 c1
podman run -d --name c1 alpine sleep 600
podman exec c1 ls /merk.txt        # No such file or directory
```

Uitleg. `podman rm` vernietigt de container **én** zijn schrijflaag. De nieuwe `c1` vertrekt weer van de exacte toestand van de image. Gegevens die je wilt bewaren, moeten dus de container uit — het onderwerp van lab 06.

PODMAN
Waarom `-t 0`? `podman rm -f` begint met een beleefd stopverzoek (`SIGTERM`), wacht **10 seconden** en slaat dan pas toe. Docker doodt meteen. Omdat `sleep` `SIGTERM` negeert (lab 03), zou je zonder `-t 0` tien seconden naar de waarschuwing `StopSignal SIGTERM failed to stop container … resorting to SIGKILL` zitten staren. Dat is geen bug: Podman vertelt je zo dat je applicatie niet netjes afsluit.

Stap 7 — Cgroups, of de verbruikslimiet

```
podman run -d --name limiet --memory=128m --memory-swap=128m alpine sleep 600
podman stats --no-stream limiet
```

Observeer de kolom `MEM USAGE / LIMIT`: `471kB / 134.2MB` — en niet het totale RAM van je machine.

Vergelijk met een container zonder limiet:

```
podman stats --no-stream waker
```

Observeer dat de getoonde limiet het totale RAM is… **van de WSL-VM**, bijvoorbeeld `7.7GB` op een pc met 16 GB.

Uitleg. Zonder `--memory` mag een container al het beschikbare geheugen opsouperen. De namespace beschermt hier tegen niets: het is de cgroup die het plafond legt. Faalt deze stap met `OCI runtime error: … cgroup …`, dan is `systemd` niet actief in je WSL (stap 0).

> **→ WINDOWS / WSL**
> WSL 2 krijgt standaard maar **50 % van het RAM** van Windows te zien (en op oudere versies hoogstens 8 GB). Je stelt het bij in `%UserProfile%\.wslconfig` (`[wsl2]`, dan `memory=12GB`). Als een container op een Windows-machine "geheugen tekortkomt", is de limiet die telt dus vaak deze — niet die van de container.

Stap 8 — `inspect`, de bron van waarheid

```
podman inspect waker | head -n 30
```

Dat is veel tekst. Richt je met een *Go-template* op wat je nodig hebt:

```
podman inspect --format '{{.State.Status}}' waker
podman inspect --format '{{.Config.Image}}' waker
podman inspect --format '{{json .Config.Cmd}}' waker
podman inspect --format '{{.NetworkSettings.IPAddress}}' waker
```

Observeer achtereenvolgens `running`, `docker.io/library/alpine:latest`, `["sleep","600"]`… en **een lege regel** voor het IP-adres.

```
podman exec waker ip -4 addr show eth0
```

Observeer dat de container tóch een interface `eth0` heeft, met **hetzelfde IP-adres als je WSL** (`172.2x.x.x`).

Uitleg. In rootless-modus mag een gewone gebruiker geen netwerkbrug aanmaken. Podman gebruikt daarom `pasta`, een vertaler in userspace die het adres van de host in de container *kopieert*; een eigen "container-IP" is er dus niet. In lab 07 komen we hierop terug. Onthoud voorlopig dat die lege waarde geen fout is, en dat `--network podman` je wél een echte brug geeft, met een IP `10.88.0.x`:

```
podman run -d --network podman --name brug alpine sleep 600
podman inspect --format '{{.NetworkSettings.Networks.podman.IPAddress}}' brug
```

Observeer `10.88.0.2`. Vergelijk nu met de metadata van de **image**:

```
podman image inspect --format '{{json .Config.Cmd}}' alpine
podman image inspect --format '{{.Architecture}}/{{.Os}}' alpine
```

Observeer dat ook de image een standaardcommando meedraagt (`["/bin/sh"]`) — dat jouw `sleep 600` bij de `run` overschreven heeft — en `amd64/linux`.

Uitleg. `podman inspect` werkt op **alle** objecten (container, image, volume, netwerk) en toont de echte toestand, zonder opsmuk. Als documentatie en werkelijkheid van elkaar afwijken, heeft `inspect` gelijk.

Stap 9 — De CLI: lange vorm, korte vorm… en `docker`

```
podman container ls -a
podman ps -a
podman image ls
podman images
```

Observeer dat de uitvoer twee aan twee identiek is.

```
podman container ls --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
```

Doe je nu even voor als Docker:

```
alias docker=podman
docker ps
docker images
```

Observeer dat alles werkt. Wil je de alias permanent maken: `echo 'alias docker=podman' >> ~/.bashrc`. Op Ubuntu doet het pakket `podman-docker` hetzelfde (het levert een `docker`-binary dat `podman` aanroept).

Uitleg. `--format` aanvaardt een Go-template en maakt de uitvoer bruikbaar in scripts — een stuk betrouwbaarder dan de standaardtabel met `awk` uit elkaar te knippen. En die alias: CLI-compatibiliteit is een belofte van Podman, en precies daardoor kun je elke Docker-tutorial gewoon volgen.

Opruimen

```
podman rm -f -t 0 waker c1 c2 limiet brug
podman ps -a
```

De `Exited`-container uit stap 2 staat er nog. Verwijder hem bij naam:

```
podman ps -a --filter status=exited --format '{{.Names}}'
podman rm <naam>
```

En wil je de ruimte van de Debian-image terug — die komt niet meer van pas:

```
podman images
podman rmi debian          # alpine houden we voor de volgende labs
```

VALKUIL
je komt overal `podman container prune`, `podman image prune -a` en `podman system prune -a` tegen. Die commando's verwijderen niet "wat je net gedaan hebt", maar **alles wat niet in gebruik is** — de images en containers van je andere projecten dus ook. Verwijder altijd bij naam. `prune` behandelen we grondig in lab 10.

Wat je nu moet kunnen beweren

- De kernel die je in een container ziet, is die van de host — hier: die van WSL 2.
- Het proces van een container staat in de `ps` van de host, onder **jouw** gebruiker — je hebt het gezien, PID en al.
- De `root` van een rootless container is een projectie van jouw UID: `podman unshare cat /proc/self/uid_map` bewijst het.
- Wat een container schrijft, bereikt de image niet, en de andere containers evenmin.
- `podman rm` vernietigt gegevens; `podman stop` niet. `podman rm -f` wacht 10 s zonder `-t 0`.
- Zonder `--memory` is de enige limiet het RAM van de WSL-VM.
- `podman inspect --format` is je eerste diagnosereflex — en een leeg IP in rootless-modus is normaal.